Convergence Is Not Correctness: Verifying Mergeable Replicated Data Types

ANONYMOUS AUTHOR(S)

CCS Concepts: • **Theory of computation** → **Program verification**; • **Software and its engineering** → *Software libraries and repositories.*

Additional Key Words and Phrases: CRDT, MRDT, RA-linearizability, convergence, sequential specification, verification

ACM Reference Format:
Anonymous Author(s). 2027. Convergence Is Not Correctness: Verifying Mergeable Replicated Data Types. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 1 (October 2027), 24 pages.

1 INTRODUCTION

Modern decentralized applications replicate data across multiple locations to improve availability, reduce access latency, and tolerate failures. Replicas must often continue operating when they cannot communicate with one another. They therefore apply updates locally and reconcile their states when communication becomes possible again. During a disconnection, different replicas may perform conflicting updates, which must be resolved when their states are merged.

Mergeable Replicated Data Types (MRDTs) [Irmin 2021] follow the model of a decentralized version control system such as Git [Git 2021]. Each update creates a new *version*, and two versions can be reconciled using a user-defined merge function. This function takes the two versions and their Lowest Common Ancestor (LCA), the latest version from which both branches diverged. When updates from different branches conflict, a *conflict resolution policy* rc determines which operations must be ordered relative to one another. Conflict-Free Replicated Data Types (CRDTs) [Shapiro et al. 2011a,b] address the same problem but do not use an LCA. Instead, they maintain causal context as metadata within the state. For example, an observed-removed set must retain information about removed elements to prevent concurrent updates from reintroducing elements that have already been removed. Such information cannot always be discarded without global coordination. MRDTs can rely on the LCA to provide the necessary causal context, avoiding some of this metadata and simplifying their implementations.

This flexibility also makes MRDTs harder to reason about. CRDTs restrict states to a join-semilattice, updates to monotonic functions, and merging to the lattice join. These restrictions simplify their correctness arguments. MRDTs impose no such restrictions. Versions can be merged at different points in an execution and in different orders, so an incorrect implementation may fail only under a particular sequence of merges.

Two properties are important for an MRDT implementation. First, replicas that have observed the same updates should reach the same state. This property is called convergence. Second, the behaviour observed by clients must agree with the sequential specification of the data type. Convergence alone does not guarantee this: replicas may agree on a state even when the state violates the intended behaviour of the data type. A correct implementation must satisfy both properties.

Replication-Aware Linearizability (RA-linearizability) is used to reason about this second property. It requires the state of a replica to be explained by a sequential ordering of its update operations, with the resulting behaviour allowed by the sequential specification. However, existing definitions have limitations. Wang et al. [2019] use specifications that include metadata from the

2027. 2475-1421/2027/10-ART1
https://doi.org/

implementation, while Soundarapandian et al. [2025] reason about the implementation's operations without a separate sequential specification. As a result, the specification may depend on the implementation being verified. This makes it difficult to use the same specification to verify different implementations of the same data type. We discuss this problem in §2.3.

There is also a problem with requiring a single global order of all update events. Existing definitions use such an order to explain the state of each version by considering the events visible to it. However, a global order that satisfies all these requirements may not exist, even when the implementation and its sequential specification are correct. Requiring one order for the entire execution can therefore rule out valid executions.

We address these limitations with a definition of RA-linearizability that considers each version separately (§4.1). For each version, we require a linearization of its visible update events that respects the visibility relation and the required ordering between conflicting operations. The resulting sequence, together with the query result, must be allowed by the sequential specification. The orders chosen for different versions need not come from one global order. We show that they are compatible and that all valid linearizations of the same version produce the same state. These results allow our definition to establish strong eventual consistency as well as correctness against the sequential specification.

This definition also calls for a different verification technique (§5). Existing methods construct a linearization of a merged version from the linearizations of the versions being merged. Such methods rely on the existence of a global order, which our definition does not require. We instead derive five verification conditions on merge and prove that they ensure that every version has the state obtained by applying its update events in a valid linearization. The details of these conditions and their proof are given in §5.

Establishing convergence does not by itself guarantee conformance to the sequential specification. The implementation's update operations may still produce states that the specification does not allow. In §6, we show how to establish this remaining property. We prove that the state property established by our verification conditions, together with sequential conformance, implies RA-linearizability.

Our contributions are as follows:

- We define RA-linearizability for MRDTs against a sequential specification of the data type, rather than the implementation. Our definition requires a linearization for each version instead of a single global order. We show that these linearizations are compatible across versions and that the definition guarantees strong eventual consistency (§4.1).
- We develop a verification technique that avoids constructing linearizations of merged versions. Instead, we derive five verification conditions on merge and show that they ensure that every version has the state obtained from a valid linearization of its update events (§5).
- We separate convergence from conformance to the sequential specification. We establish the required state property using the five verification conditions and prove sequential conformance separately. A theorem combines these results to establish RA-linearizability (§6).
- We use the OR-set as a running example to illustrate the limitations of existing definitions and verification methods and to demonstrate how our approach addresses them.

The remainder of the paper is organized as follows. §2 introduces the MRDT model and discusses the limitations of existing definitions of RA-linearizability. §3 presents the operational semantics of the Git-like replicated store. §4.1 defines the linearization relation and RA-linearizability. §5 presents the verification conditions and the adequacy theorem. Finally, §6 explains how the state property established by the verification conditions is combined with sequential conformance.

2 MOTIVATION

2.1 The MRDT Model

An MRDT system model resembles a distributed version control system such as Git [Git 2021]. The data is replicated across replicas $r_1, r_2, \ldots \in \mathcal{R}$, and each replica keeps a *version* of the state. Replicas operate concurrently, performing updates without synchronizing with each other, and periodically exchange their states through a process called *merge*. Since operations happen concurrently at several replicas, conflicts may arise, and the merge must resolve them in a consistent manner. An object has a type $\tau \in Type$ with signature $\langle O_\tau, Q_\tau, Val_\tau \rangle$, which specifies its update operations, its query operations and their return values.

Definition 2.1 (MRDT implementation). An MRDT implementation for a data type τ is a tuple

$$\mathcal{D}_\tau = \langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{merge}, \mathsf{query}, \mathsf{rc} \rangle,$$

where

- Σ is the set of states and $\sigma_0 \in \Sigma$ is the initial state.
- $\mathsf{do} : \Sigma \times \mathcal{T} \times \mathcal{R} \times O_\tau \to \Sigma$ implements all update operations in O_τ, where $\mathcal{T}$ is the set of timestamps.
- $\mathsf{merge} : \Sigma \times \Sigma \times \Sigma \to \Sigma$ is a three-way merge function.
- $\mathsf{query} : \Sigma \times Q_\tau \to Val_\tau$ implements all query operations in Q_τ, returning a value in Val_τ.
- $\mathsf{rc} \subseteq O_\tau \times O_\tau$ is the conflict resolution policy to be followed for concurrent update operations.

An MRDT $\mathcal{D}_\tau$ thus provides implementations of do, merge and query, which the data store invokes as clients perform operations and replicas exchange states. A client request to perform an update operation $o \in O_\tau$ at a replica r triggers the call $\mathsf{do}(\sigma, t, r, o)$. This takes as input the current state σ of r and a unique timestamp t, and produces an updated state which is then installed at r. The data store ensures that timestamps are unique across all operations, which can be achieved through e.g. Lamport timestamps [Lamport 1978].

A replica can also receive a version from another replica, which is combined with its own using merge. The merge function is called with the current states of both replicas together with their lowest common ancestor, which represents the most recent common state from which the two diverged. Clients read the state using query, which takes a state and a query operation and produces a return value. Note that a query operation cannot change the state at a replica.

While merging, it may happen that conflicting (i.e. non-commutative) update operations have been performed on the two states, and rc says how to resolve them. It relates operations, and $o_1 \xrightarrow{\mathsf{rc}} o_2$ is to be read as follows: whenever two *concurrent* non-commutative operations o_1 and o_2 are performed on different replicas and the resulting states are merged, the merged state should be the one obtained by performing o_1 first and o_2 afterwards[1]. Because it relates operations rather than operation instances (i.e. events), the policy is fixed once by the datatype. Since the order of two commuting operations makes no difference to the state that results, it orders only the non-commuting pairs.

The OR-set. Fig. 1 gives the observed-remove set [Shapiro et al. 2011b] as an instance of Definition 2.1. It uses an add-wins conflict-resolution strategy, so that when an element is added on one replica and concurrently removed on another replica, the addition wins. This implementation is close to the operation-based CRDT implementation of the OR-set [Shapiro et al. 2011a]. The state is

[1]This is a slight simplification, as rc is only enforced for operations which are not followed by another non-commutative operation happening later on the same replica; see definition 4.1

a set of element-timestamp pairs, with $\mathbb{E}$ being the set of elements and $\mathcal{T}$ the set of timestamps. An add_a records the element together with the unique timestamp of the operation. A rem_a removes every entry for a in the set. An element is in the set when some entry for it is present.

1: $\Sigma = \mathcal{P}(\mathbb{E} \times \mathcal{T})$
2: $O = \{\mathsf{add}_a, \mathsf{rem}_a \mid a \in \mathbb{E}\}$
3: $Q = \{\mathsf{rd}_a \mid a \in \mathbb{E}\}$
4: $\sigma_0 = \{\}$
5: $\mathsf{do}(\sigma, t, _, \mathsf{add}_a) = \sigma \cup \{(a, t)\}$
6: $\mathsf{do}(\sigma, _, _, \mathsf{rem}_a) = \sigma \setminus \{(a, i) \mid (a, i) \in \sigma\}$
7: $\mathsf{merge}(\sigma_\top, \sigma_1, \sigma_2) = (\sigma_\top \cap \sigma_1 \cap \sigma_2) \cup (\sigma_1 \setminus \sigma_\top) \cup (\sigma_2 \setminus \sigma_\top)$
8: $\mathsf{query}(\sigma, \mathsf{rd}_a) = \exists i.\ (a, i) \in \sigma$
9: $\mathsf{rc} = \{(\mathsf{rem}_a, \mathsf{add}_a) \mid a \in \mathbb{E}\}$

Fig. 1. The OR-set, in the MRDT implementation model of Soundarapandian et al. [2025]

The merge keeps the entries of the LCA that survived in both branches, and adds the entries that are new in either branch. The relation rc orders rem_a before add_a of the same element, which is what produces the add-wins behaviour. The merge implementation along with the fact that each add operation adds an element tagged with a unique timestamp ensures the rc relation, as a rem_a operation will not be able to remove the tagged element added by a concurrent add_a operation, and this tagged element would be survive in the final merged version. All other pairs of operations commute, namely two additions, two removals, and an addition and a removal of different elements. Thus, rc does not order them.

An execution of the store builds a graph of versions. Each version is created either by applying an operation to an existing version, or by merging two of them, or by forking a new replica. Each replica has a *head*, the most recent version it holds. Every update operation performed in an execution is an *event* of the form (t, r, o), which records the operation o, the replica r which performed it and its unique timestamp t. Each version carries the set of events which led to it, and we write $E(v)$ for the event set of a version v. We write $a \xrightarrow{\mathsf{vis}} b$ when the event a was visible at the replica which performed b.

2.2 Correctness Against a Natural Specification

The OR-set of Fig. 1 is supposed to behave like a set. If a client adds a, then removes a, and then reads, the set should not contain a. The client does not see the timestamps with which an element was added, and knows nothing about merges. So the specification against which this implementation must be checked is just an ordinary sequential set, whose state σ is a set of elements, initially empty, and whose operations behave as follows.

$$\sigma \xrightarrow{\mathsf{add}_a} \sigma \cup \{a\} \qquad \sigma \xrightarrow{\mathsf{rem}_a} \sigma \setminus \{a\} \qquad \sigma \xrightarrow{\mathsf{rd}_a \Rightarrow (a \in \sigma)} \sigma$$

The operations written over the arrows are the *labels* of this specification. A label is an operation, and for a query it carries the return value as well. In fact, these labels exactly correspond to the set of update operations O_τ and query operations (Q_τ) implemented by an RDT. An add_a puts a into the set and a rem_a removes a from it, while a rd_a leaves the state unchanged and returns *true* if a is present in the set, and *false* otherwise. This specification is not related to any actual implementation of the datatype, and we call it the *natural specification* of the set. It is the same kind of specification that classical linearizability uses [Herlihy and Wing 1990], where a concurrent object is judged against the sequential specification of the datatype it implements.

Connecting an RDT implementation to such a specification is the goal of *replication-aware linearizability* [Wang et al. 2019], or RA-linearizability for short. The operations of an RDT are performed at different replicas and the resulting states are merged afterwards, so there is no single order in which the operations actually happen during an execution. However, to *linearize* a set of operations in an execution is to arrange them in a sequence which should be justified by the execution and which must also be admitted by the specification. This notion is formalized by both the prior works in this area (Wang et al. [2019] for CRDTs and Soundarapandian et al. [2025] for MRDTs) in the following fashion: Given an execution, there is a global lo order, called the linearization order, which orders all the non-commutative update events. It has two kinds of edges. One is the vis edge (corresponding to the $\xrightarrow{\mathsf{vis}}$ relation) and orders non-concurrent events, and the other is rc, which orders concurrent events. Then, for every query event, we take the update events visible to it, linearize them according to the global lo order, and ask that the resulting sequence of labels of these events, followed by the label of the query event, belongs to the specification.

Note that the above definition comes from Soundarapandian et al. [2025]; Wang et al. [2019] build a single sequence of update events instead of maintaining an lo order, and take sub-sequences corresponding to the update events visible to a query event. Both these representations are identical, and we will often switch between the two to aid readability. The definition of RA-linearizability closely follows upon the classical definition of linearizability, replacing the real-time order (i.e. happens-before ordering) between operation invocations of the latter with the vis and rc relations of the former. However, the issue arises when trying to relate the linearization to the sequential specification, and this is one of the primary concerns of this paper.

2.3 Problems with the Current Definitions of RA-Linearizability

The primary problem with existing works on RA-linearizability is that they do not consider a natural sequential specification. Specifically, Wang et al. [2019] consider sequential specifications which are very close to the actual implementation, while [Soundarapandian et al. 2025] does not consider any sequential specification at all, and just shows linearization with respect to the operations of the implementation itself. To illustrate this, consider the specification of the set datatype from Wang et al. [2019] whose abstract state σ is a set of element-timestamp pairs, and the transitions are as given below:

$$\sigma \xrightarrow{\mathsf{add}(a,k)} \sigma \cup \{(a,k)\} \qquad \sigma \xrightarrow{\mathsf{rem}(S)} \sigma \setminus S$$

$$\sigma \xrightarrow{\mathsf{readIds}(a) \Rightarrow S} \sigma \quad [\, S = \{(a,k) \mid (a,k) \in \sigma\}\,] \qquad \sigma \xrightarrow{\mathsf{rd}_a \Rightarrow B} \sigma \quad [B = \exists k.\ (a,k) \in \sigma]$$

Notice that add_a now becomes $\mathsf{add}(a,k)$, where k is a timestamp, while rem_a is actually split into a query $\mathsf{readIds}(a)$ collecting the pairs containing a into a set S, followed by an update $\mathsf{rem}(S)$ deleting exactly those pairs. This is significantly different from the natural specification of a set given in §2.2. The splitting of rem_a into a query and an update is actually an artifact of the operation-based CRDT model considered by Wang et al. [2019], where an operation is a generator followed by an effector.

The specification changes with the implementation. A datatype should have one sequential specification, irrespective of the implementation. The specifications proposed in Wang et al. [2019] do not allow this as we demonstrate now using two different RDT implementations of the set datatype.

The first is the one in Fig. 1. Consider an execution at a single replica r_1, which adds a with timestamp 1, adds a again with timestamp 2, and then removes a. The resulting three update events will then be linearized in the above order (due to the $\xrightarrow{\mathsf{vis}}$ relation among them), resulting in the

following sequence of labels, which we write in the alphabet of lablels used in Wang et al. [2019]'s specification, i.e. replacing add_a by the timestamp tagged pair which is added, and rem_a by the pair that it removes.

$$\mathsf{add}(a,1)\cdot\mathsf{add}(a,2)\cdot\big(\mathsf{readIds}(a)\Rightarrow\{(a,1),(a,2)\}\big)\cdot\mathsf{rem}(\{(a,1),(a,2)\}),\tag{1}$$

This sequence belongs to Wang et al. [2019]'s specification of the set data-type. Now, consider another RDT implementation of the set datatype, namely the efficient OR-set of Soundarapandian et al. [2025]. In this implementation, the state is again a set of element-timestamp pairs, but a timestamp is itself now a pair of replica identifier and a natural number whose value is derived using a per-replica counter. An add_a operation issued at replica r whose counter value is c now checks whether the state contains a pair of the form $(a,\langle r,c'\rangle)$, and *replaces* such an pair with $(a,\langle r,c\rangle)$. rem_a is exactly the same as its counterpart in the original OR-set implementation. Now, consider the same execution as before, which would again be linearized in the same order, but the labels now will be different:

$$\mathsf{add}(a,\langle r_1,1\rangle)\cdot\mathsf{add}(a,\langle r_1,2\rangle)\cdot\big(\mathsf{readIds}(a)\Rightarrow\{(a,\langle r_1,2\rangle)\}\big)\cdot\mathsf{rem}(\{(a,\langle r_1,2\rangle)\}),\tag{2}$$

In particular, since the second add_a replaced the pair generated by the first add_a, rem_a only needs to remove the entry with timestamp $\langle r_1,2\rangle$.

Sequence (2) is not admitted by Wang et al. [2019]'s specification, and the read performed by remove is where it fails. After the two add operations, the specification state holds both pairs, so the answer to $\mathsf{readIds}(a)$ is both of them, while (2) answers with the entry of timestamp $\langle r_1,2\rangle$ alone. The specification written for the first implementation therefore rejects the second, which needs a specification of its own. We note that the natural specification can explain this execution in both implementations, as the label sequence would just be $\mathsf{add}_a\cdot\mathsf{add}_a\cdot\mathsf{rem}_a$, with the abstract state being the empty set at the end, which would correspond to the concrete state also being empty at the end of the execution in both the implementations.

The natural specification rejects a correct implementation. Suppose we refuse the specification of Wang et al. [2019], because it is tied to the implementation, and keep the natural specification of §2.2. However, we now show that the definition of RA-Linearizability itself would prevent us from showing even the original OR-Set implementation to be correct with respect to the natural specification. In particular, the execution of Fig. 2 admits no linearization with respect to the natural specification. The issue lies in the fact that the definition of RA-linearizability requires a global linearization order among all the update events to explain every state generated during an execution. In this execution, there are three replicas, namely p, q and r, and four events all applied to one element a:

$$A_p=(1,p,\mathsf{add}_a),\qquad A_q=(2,q,\mathsf{add}_a),$$
$$R_p=(3,p,\mathsf{rem}_a),\qquad R_q=(4,q,\mathsf{rem}_a),$$

Recall the definition of an event: the first component of each triple is the timestamp of the event and the second the replica at which it was performed.

Time runs downwards in Fig. 2 and each replica is a vertical line. Replicas p and q concurrently perform add_a from v_0, giving v_1 and v_2. Replica r then merges twice, first with p and then with q, to get v_3. Each of p and q then performs a rem_a, having seen only its own addition, so v_4 and v_5 are both empty. Each now merges in v_3, which brings the *other* replica's entry, giving $v_6=\{(a,2)\}$ and $v_7=\{(a,1)\}$. Finally q merges p's head v_6 into its own head v_7, giving v_8.

Every one of these states is correct for what the replica has seen, in the sense that the state can be explained using a linearization of the update operations leading to that state, the linearization would obey both the vis relation between non-concurrent events and the rc relation between

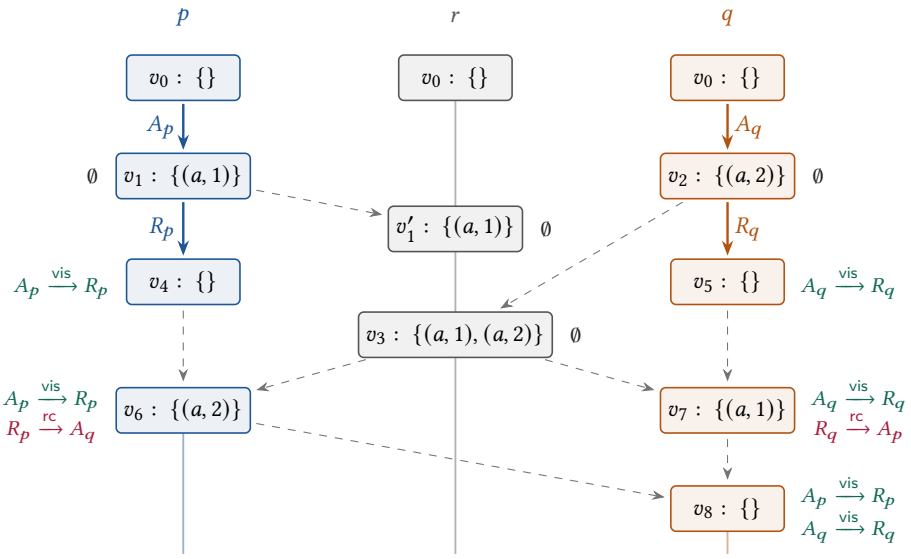


Fig. 2. An execution of OR-Set which violates the natural specification of the set datatype as per the existing definition of RA-Linearizability. Time runs downwards and each replica is a vertical line. Solid arrows apply an event and dashed arrows are merge edges. Each box holds a version and its state, with the linearization relation lo of Soundarapandian et al. [2025] on that version's own event set beside it, each edge marked as coming from vis or from rc.

concurrent events, and it would also belong to the natural specification of the set datatype. For example, consider the version v_6, which can be explained by the linearization $A_p \cdot R_p \cdot A_q$ as performing these operations would exactly result in the state $\{(a, 2)\}$. Further, this linearization belongs to the natural specification, as a read_a query on the state would return *true*, resulting in the sequence $\mathsf{add}_a \cdot \mathsf{rem}_a \cdot \mathsf{add}_a \cdot \mathsf{read}_a \Rightarrow \mathit{true}$. In Fig. 2, we also show the lo relations caused by vis and rc edges alongside each version. Note that for v_8, the rc relation from R_p to A_q does not cause an lo relation between them due to the remove event R_q which occurs after A_q in the same replica, and effectively absorbs the A_q event.

The problem here is that RA-Linearizability asks for a single linearization of all update events in the entire execution which should be projected to the events of each version to justify that version and such a linearization does not exist in this execution. For instance, consider the linearization $A_p \cdot R_p \cdot A_q \cdot R_q$ which preserves the lo relation. Its projection onto the events of v_6 is $A_p \cdot R_p \cdot A_q$ which agrees with the state at v_6. But its projection onto the events of v_7 is $A_p \cdot A_q \cdot R_q$, which does not justify the state at v_7 as these sequence of operations would lead to the empty state $\{\}$. Every other candidate sequence would also fail in a similar way. We note that this execution can be shown to be correct with respect to Wang et al. [2019]'s specification, as for the projection $A_p \cdot A_q \cdot R_q$ for v_7, the sequence of labels would be $\mathsf{add}(a, 1) \cdot \mathsf{add}(a, 2) \cdot \mathsf{rem}((a, 2))$, with the label of R_q coming from the elements that R_q read when it was executed. This corresponds to the state $\{(a, 1)\}$ which matches the state of v_7.

To summarize, we have a situation where the original OR-Set implementation cannot be shown to be correct with respect to the natural set specification, while the efficient OR-Set implementation cannot be shown correct with respect to Wang et al. [2019]'s specification.

The Repair, informally. The root cause of the issue does not lie in the specifications, but rather in the definition of RA-Linearizability itself, which forces every version to be explained by projecting from a single global linearization of all the update events. The solution that we propose is to remove the requirement of having a single global linearization. Instead, for every version, we ask for a linearization of update events visible to that version, which must obey the vis and rc relations projected to the events of that version. Then, for such a linearization of update events followed by a query operation, we check whether the resulting sequence is admitted by the specification. The execution of Fig. 2 can now be showed to be correct with respect to the natural set specification. In particular, for the version v_7, we consider the linearization $A_q \cdot R_q \cdot A_p$ which leads to the concrete state $\{(a, 1)\}$ of v_7, and this also corresponds to the abstract state obtained after the sequence $\mathsf{add}_a \cdot \mathsf{rem}_a \cdot \mathsf{add}_a$ of the natural specification. Every version can be similarly shown to be correct with respect to the natural set specification. We give a formal definition in §4.

We also propose a new verification procedure for proving RA-Linearizability of MRDTs. The procedure is based on the bottom-up linearization approach proposed in Soundarapandian et al. [2025]. In this approach, the linearization of a merged version is built out of the linearizations of the two versions being merged. To do so, Soundarapandian et al. [2025] proposed rules which take out the last event out of the linearizations of one of the two versions being merged, and require the linearization of the merged state to have that same last event. This becomes possible due to the presence of a global linearization order, which ensures that the last event of the merged state must be the last event of either of the two versions. However, with our proposed new definition of RA-Linearizability, this no longer holds true.

To illustrate this, consider the merge in Fig. 2 which produces v_8 from v_6 and v_7. It has all four events, and on that set of event both rc edges are cancelled, R_q absorbing A_q and R_p absorbing A_p, so lo keeps only $A_p \xrightarrow{\mathsf{vis}} R_p$ and $A_q \xrightarrow{\mathsf{vis}} R_q$. The states of v_6 and v_7 are $\{(a, 2)\}$ and $\{(a, 1)\}$, which correspond to the linearizations $A_p \cdot R_p \cdot A_q$ and $A_q \cdot R_q \cdot A_p$ respectively. The state of v_8 is $\{\}$, so its linearization has to end with R_p or R_q, a removal being what takes every entry of a out. To build it from the two versions being merged, one of the two linearizations of v_6 or v_7 would have to end with a remove event (i.e. either R_p or R_q), which could then be placed last in the linearization of the merge. However, neither of them does, as both sequences end with an add event, and hence bottom-up linearization would fail.

We rectify this by proposing a novel merge re-distribution strategy, which rewrites the states being merged in such a way that the last event in the linearization of the merged state is guaranteed to be the last event in one of the two states being merged. This allows bottom-up linearization to peel off the last event, and continue with the rest of the linearization. We describe our new strategy in detail in §5.

3 OPERATIONAL SEMANTICS

This section describes the semantics of the replicated data store in which the MRDT implementation runs. The model is similar to that of Soundarapandian et al. [2025]. The semantics of the store defines all possible executions of an MRDT implementation. It is parameterised by an implementation $\mathcal{D}_\tau = \langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{merge}, \mathsf{query}, \mathsf{rc} \rangle$ of type $\tau = \langle O_\tau, Q_\tau, Val_\tau \rangle$, and is represented by a labelled transition system $\mathcal{S}_\mathcal{D} = (C, \rightarrow)$. Each configuration in C maintains a set of *versions*, where each version is created by applying an operation to an existing version, or by merging two versions, or by forking a new replica. Each replica is associated with a *head* version, which is the most recent version it has seen. Formally, each configuration C in C is a tuple $\langle S, H, P, \mathsf{vis} \rangle$, where

- $S : \mathsf{Version} \rightharpoonup \Sigma \times \mathcal{P}(\mathcal{E})$ is a partial function mapping each version to its state together with the set of events that led to it. A version is *active* when it lies in the domain of S, and we

write $E(v)$ for the event set of v. Each event is an instance of an update operation, uniquely identified by its timestamp, so $\mathcal{E} = \mathcal{T} \times \mathcal{R} \times O_\tau$. $\mathsf{time}(e)$ and $\mathsf{rep}(e)$ gives the timestamp and replica ID of an event e respectively.

- $H : \mathcal{R} \rightharpoonup \mathsf{Version}$ maps each replica to its head version. A replica is *active* when it lies in the domain of H.
- $P : \mathsf{Version} \to \mathsf{List}(\mathsf{Version})$ gives the parents of each version. A version created by applying an event has one parent, and a version produced by a merge has two parents. These edges form the version graph. We say that v_1 is an *ancestor* of v_2 when v_2 is reachable from v_1 by following them. We write $v_1 \preceq v_2$ when v_1 is an ancestor of v_2 or equal to it. A version $v_\top$ is a *lowest common ancestor* (LCA) of v_1 and v_2 when $v_\top \preceq v_1$ and $v_\top \preceq v_2$, and for every w with $w \preceq v_1$ and $w \preceq v_2$, $w \preceq v_\top$.
- vis is a strict partial order on events. If $a \xrightarrow{\mathsf{vis}} b$, then $\mathsf{time}(a) < \mathsf{time}(b)$. Distinct events performed at the same replica are comparable under vis.

There are four transition rules. Fork creates a new replica r' from an active replica r. It installs a fresh version v at r', and its parent is $H(r)$. The new version holds the same state and event set as $H(r)$. Query reads the state at the head version of a replica and returns the query result without changing its state. The above two rules are the same as that of Soundarapandian et al. [2025], so we skip their inference rules. The two rules which are important are given in Fig. 3.

[Apply]

$$\frac{\begin{array}{c} e = (t, r, o) \\ o \in O_\tau \quad r \in \mathsf{dom}(H) \quad v \notin \mathsf{dom}(S) \quad S(H(r)) = (s, E) \quad \forall e' \in \bigcup \mathsf{range}(S).\, \mathsf{time}(e') \neq t \quad \forall e' \in E.\, \mathsf{time}(e') < t \\ S' = S[v \mapsto (\mathsf{do}(s, t, r, o),\ E \cup \{e\})] \quad H' = H[r \mapsto v] \quad P' = P[v \mapsto [H(r)]] \quad \mathsf{vis}' = \mathsf{vis} \cup (E \times \{e\}) \end{array}}{\langle S, H, P, \mathsf{vis}\rangle \xrightarrow{\mathsf{apply}(t,r,o)} \langle S', H', P', \mathsf{vis}'\rangle}$$

[Merge]

$$\frac{\begin{array}{c} r_1, r_2 \in \mathsf{dom}(H) \quad v \notin \mathsf{dom}(S) \quad v_\top = \mathsf{LCA}(H(r_1), H(r_2)) \quad S(v_\top) = (s_0, E_0) \quad S(H(r_1)) = (s_1, E_1) \\ S(H(r_2)) = (s_2, E_2) \quad S' = S[v \mapsto (\mathsf{merge}(s_0, s_1, s_2),\ E_1 \cup E_2)] \quad H' = H[r_1 \mapsto v] \quad P' = P[v \mapsto [H(r_1), H(r_2)]] \end{array}}{\langle S, H, P, \mathsf{vis}\rangle \xrightarrow{\mathsf{merge}(r_1,r_2)} \langle S', H', P', \mathsf{vis}\rangle}$$

Fig. 3. The two rules of the store which matter here. Fork and Query are exactly as in Soundarapandian et al. [2025] and are omitted.

Apply applies an update operation o at a replica r, generating a new event e whose timestamp is different from that of all events generated so far and greater than the timestamp of every event in $E(H(r))$. $\bigcup \mathsf{range}(S)$ denotes the events witnessed across all versions. A new version is created whose state is obtained by applying o to the current state of r, and whose event set is $E \cup \{e\}$. Every event $e' \in E$ is made visible to e.

Merge takes two replicas r_1 and r_2, applies the merge function to the states of their head versions and the state of the lowest common ancestor $v_\top$, to generate a new version v which becomes the head version of r_1. The new version has the two head versions as its parents. Its event set is the union of the two, and the visibility relation does not change, since a merge generates no event.

The initial configuration of $\mathcal{S}_\mathcal{D}$ is $C_0 = \langle S_0, H_0, P_0, \emptyset\rangle$, which consists of a single replica r_0 and a single version v_0. Here $H_0 = [r_0 \mapsto v_0]$, and $S_0 = [v_0 \mapsto (\sigma_0, \emptyset)]$ where σ_0 is the initial state of $\mathcal{D}_\tau$ and v_0's event set is empty, since no updates has been performed yet. The initial version has no parents, so $P_0 = [v_0 \mapsto [\,]]$, and vis is an empty relation. An *execution* of $\mathcal{S}_\mathcal{D}$ is a finite

sequence of transitions $C_0 \xrightarrow{t_1} C_1 \xrightarrow{t_2} \cdots \xrightarrow{t_n} C_n$ starting from C_0, and $[\![\mathcal{S}_{\mathcal{D}}]\!]$ denotes the set of all executions of $\mathcal{S}_{\mathcal{D}}$.

There are two facts about reachable configurations for all data types, and we use them throughout. The first is that the event set of every active version v is causally closed, so $a \in E(v)$ whenever $b \in E(v)$ and $a \xrightarrow{\mathsf{vis}} b$. The second is about the LCA version of the merge.

Lemma 3.1 (LCA). *In any reachable configuration, if $v_\top$ is a lowest common ancestor of v_1 and v_2, then $E(v_\top) = E(v_1) \cap E(v_2)$.*

Thus, the events of the LCA are exactly those applied at both merging versions.

Merges with several potential LCAs. Two versions may have several common ancestors and no LCA. Soundarapandian et al. [2025] call a common ancestor w of v_1 and v_2 a *potential* LCA when there is no other common ancestor w' with $w \preceq w'$. In such cases, they merge the potential LCAs recursively to obtain the actual LCA. We use the same strategy here as well.

4 A NEW DEFINITION OF RA-LINEARIZABILITY

This section formally defines our new notion of RA-linearizability. It closely follows the definitions from Wang et al. [2019] and Soundarapandian et al. [2025], with the major difference being that it does not require a global linearization order, but rather requires individual linearization sequences for every version.

Notations. We now introduce the notation used in the rest of the paper. Applying an event $e = (t, r, o)$ to a state σ means computing $\mathsf{do}(\sigma, t, r, o)$, which we also denote by $e(\sigma)$. For a list of events π we write $\pi(\sigma)$ for the state obtained by applying the events of π to σ one after the other. Two events *commute*, written $e_1 \leftrightarrow e_2$, when $e_1(e_2(\sigma)) = e_2(e_1(\sigma))$ for every $\sigma \in \Sigma$. We write $o_1 \leftrightarrow o_2$ if every pair of events labelled o_1 and o_2 commutes. Two event are *concurrent*, written $e_1 \parallel e_2$, when neither of the events is visible to the other. If two events are related by rc, we write $e_1 \xrightarrow{\mathsf{rc}} e_2$.

4.1 Formally defining RA-Linearizability

Given a configuration C, our goal is to define a linearization order over the events in the configuration, and require states at each replica to follow this linearization order. However the execution of §2.3 showed what goes wrong when a common linearization order over all events of the configuration is defined. Hence, we now define a linearization relation parametric on a subset of events of the configuration. Every subset of events will essentially have its own linearization relation. Let $\mathcal{E}_C$ denotes the set of all events of a configuration C.

Definition 4.1 (Linearization relation). For a given configuration C, let $E \subseteq \mathcal{E}_C$ be causally closed. For events $e_1, e_2 \in E$,

$$e_1 \ \mathsf{lo}_E^C \ e_2 \iff \big(e_1 \xrightarrow{\mathsf{vis}} e_2 \wedge \neg(e_1 \leftrightarrow e_2)\big) \\ \vee \big(e_1 \parallel e_2 \wedge e_1 \xrightarrow{\mathsf{rc}} e_2 \wedge \neg(\exists e_3 \in E.\ e_2 \xrightarrow{\mathsf{vis}} e_3 \wedge \neg(e_2 \leftrightarrow e_3))\big).$$

The above definition is similar to the one in Soundarapandian et al. [2025], with the only difference being that lo_E^C is now parameterized over a causally closed subset of events E. The first clause ensures that linearization follows the visibility relation between non-commutative events. For concurrent non-commutative events, the rc relation is followed, unless there is an absorber event which absorbs the rc-later event. Notably, this absorber event is only looked for within the subset E, rather than the entire set of events in the configuration.

For example, for the execution in Fig. 2, for the version v_6 and its set of events $E = \{A_p, R_p, A_q\}$, lo_E^C would contain both $A_p\ \mathsf{lo}_E^C\ R_p$ and $R_p\ \mathsf{lo}_E^C\ A_q$. This is in spite of the absorber event R_q in the overall execution, which in fact results in dropping the $R_p\ \mathsf{lo}_E^C\ A_q$ in the version v_8. Since an absorber is looked for within an event set, the linearization order of two event sets may differ in whether a pair of events is ordered or left unordered, depending on whether an absorber event is present or not. However, they cannot order the same pair in opposite directions.

Lemma 4.2. *Let $\mathcal{D}_\tau$ satisfy NO-RC-CHAIN($\mathcal{D}_\tau$). For all $E, E' \subseteq \mathcal{E}_C$ there are no events a and b such that $a\ \mathsf{lo}_E^C\ b$ and $b\ \mathsf{lo}_{E'}^C\ a$.*

The above lemma requires the NO-RC-CHAIN($\mathcal{D}_\tau$), which is borrowed from Soundarapandian et al. [2025] and which we will formally define later in the paper, and which in this case ensures that the rc relation itself does not cause a cycle. Intuitively, the above lemma holds because lo_E^C is derived from vis or rc, and vis is a partial order and hence acyclic by construction, while rc is ensured to be acyclic by the NO-RC-CHAIN($\mathcal{D}_\tau$) condition.

We now define the notion of a linear extension which essentially linearizes lo_E^C into a sequence which orders the events of an event set E in an order obeying lo_E^C.

Definition 4.3 (Linear extension). Let C be a configuration, $E \subseteq \mathcal{E}_C$, and R a relation on events. A sequence $\pi = e_1 \cdots e_n$ is a *linear extension* of (E, R) if:

(i) Every event in E occurs exactly once in π, and π contains no events outside E.
(ii) For every $a, b \in E$ with $(a, b) \in R$, a occurs before b in π.

RA-linearizability checks whether a linear extension is allowed by a sequential specification, which we now define formally. Recall from §2.2 that an update label is an operation $o \in O_\tau$, while a query label is a pair $q \Rightarrow b$, where $q \in Q_\tau$ and $b \in Val_\tau$ is the returned value. A sequential specification is a labelled transition system over abstract states.

Definition 4.4 (Sequential specification). A *sequential specification* for a data type τ is a tuple

$$\mathsf{Spec} = \langle L, \Phi, \phi_0, \rightarrow \rangle,$$

where L is the set of labels, Φ is the set of abstract states, $\phi_0 \in \Phi$ is the initial state, and $\rightarrow \subseteq \Phi \times L \times \Phi$ is the transition relation. We write $\phi \xrightarrow{\ell} \phi'$ to denote the transition from state ϕ to state ϕ' by applying the label ℓ.

A sequence $\rho = \ell_1 \cdots \ell_n$ over L is *admitted* by Spec if the transition relation can execute it from the initial state ϕ_0. That is, there exist states $\phi_1, \ldots, \phi_n$ such that $\phi_{i-1} \xrightarrow{\ell_i} \phi_i$ for each $i = 1, \ldots, n$. We write $\rho \in \mathsf{Spec}$ when ρ is admitted by Spec. By construction, Spec is prefix-closed. If $\rho \cdot \rho'$ is admitted by Spec, then ρ is also admitted. For an admitted sequence ρ, we write $\rho(\phi_0)$ for the state it reaches. Execution of an RDT implementation generates a sequence of events, each of which contains an operation label from the specification. Given a sequence π of events, we use the notation $\mathsf{ops}(\pi)$ to project out the operation label from each event pointwise.

We now define a set of 4 pre-conditions that a datatype implementation $\mathcal{D}_\tau$ must satisfy in order for it to be RA-linearizable. Note that these are necessary conditions, in the sense that the definition of RA-Linearizability itself requires them. The first 3 out of these 4 conditions have been

directly borrowed from Soundarapandian et al. [2025].

$$\textsc{rc-non-comm}(\mathcal{D}_\tau) \triangleq \forall o_1, o_2 \in O_\tau.\ \neg(o_1 \leftrightarrow o_2) \land o_1 \neq o_2 \Leftrightarrow o_1 \xrightarrow{\mathsf{rc}} o_2 \lor o_2 \xrightarrow{\mathsf{rc}} o_1$$

$$\textsc{no-rc-chain}(\mathcal{D}_\tau) \triangleq \neg\big(\exists o_1, o_2, o_3 \in O_\tau.\ o_1 \xrightarrow{\mathsf{rc}} o_2 \xrightarrow{\mathsf{rc}} o_3\big)$$

$$\textsc{cond-comm}(\mathcal{D}_\tau) \triangleq \forall o_1, o_2, o_3 \in O_\tau.\ \big(o_1 \xrightarrow{\mathsf{rc}} o_2 \land \neg(o_2 \leftrightarrow o_3)\big) \Rightarrow o_1 \overset{o_3}{\leftrightarrow} o_2$$

$$\textsc{spec-non-comm}(\mathcal{D}_\tau, \mathsf{Spec}) \triangleq \forall o_1, o_2 \in O_\tau.\ \neg(o_1 \leftrightarrow_S o_2) \Rightarrow \neg(o_1 \leftrightarrow o_2)$$

The first condition, RC-NON-COMM, requires rc to relate exactly those pairs of operations that do not commute. This condition ensures that non-commutative operations will be linearized in a fixed order at every version. The second, NO-RC-CHAIN, disallows a chain of two rc edges. We have already used it earlier in Lemma 4.2 to ensure that rc remains acyclic. Having no chains is a stronger property than acyclicity, and it is necessary because if there is an rc-chain, there exists executions which cannot be linearized in a way that both the vis relation between non-concurrent events and rc between concurrent events is preserved. For details, we refer to Soundarapandian et al. [2025].

The third condition, COND-COMM, is about pairs whose rc edge is removed by an absorber. Once the edge is removed, either order is allowed, so both orders must produce the same result. This is ensured by the notion of conditional commutativity, which is defined as follows: Two operations o and o' conditionally commute with respect to a third operation o'', written $o \overset{o''}{\leftrightarrow} o'$, if

$$\forall \sigma \in \Sigma.\ \forall \pi \in \mathcal{E}^*.\quad o''\big(\pi(o(o'(\sigma)))\big) = o''\big(\pi(o'(o(\sigma)))\big).$$

The fourth condition, SPEC-NON-COMM is new and it connects the implementation to the specification. To define this condition, we first define the notion of commutativity in the specification. Two update labels x and y in O_τ *commute in* Spec, written $x \leftrightarrow_S y$, if swapping them does not change whether a sequence is admitted.

$$\rho \cdot x \cdot y \cdot \rho' \in \mathsf{Spec} \quad\Longleftrightarrow\quad \rho \cdot y \cdot x \cdot \rho' \in \mathsf{Spec}$$

for all label sequences ρ and ρ'.

For example, in the natural set specification, add_a and add_b commute in the specification, while add_a and rem_a do not commute. The condition SPEC-NON-COMM ensures that if two operations do not commute in Spec, they must not commute in the implementation either. This condition is necessary as our definition of RA-linearizability would allow commutative operations in the implementation to be ordered arbitrarily which may break the intended ordering from the specification's point of view. For example, one can consider a buggy OR-Set implementation which implements rem_a to be the identity function. This would make add_a and rem_a commutative in the implementation, and would make an execution where a replica performs add_a followed by rem_a RA-linearizable, because the linearization order only exists between non-commutative events, and hence this execution can be incorrectly linearized as rem_a followed by add_a, with the final state of the implementation matching the abstract state in the specification. We note it is natural to expect non-commutative operations in the Spec to remain non-commutative in the implementation.

We group these four conditions into $\mathsf{Laws}(\mathcal{D}_\tau, \mathsf{Spec})$, which we assume every implementation satisfies. We can now formally define RA-linearizability under these conditions.

Definition 4.5 (RA-linearizability). Let $\mathcal{D}_\tau$ be an MRDT satisfying $\mathsf{Laws}(\mathcal{D}_\tau, \mathsf{Spec})$. A configuration $C = \langle S, H, P, \mathsf{vis}\rangle$ of $\mathcal{S}_\mathcal{D}$ is *RA-linearizable* with respect to a sequential specification Spec if, for every active version $v \in \mathsf{dom}(S)$ and every query $q \in Q_\tau$, the following holds. Let $(s, E) = S(v)$ be the state and event set of v, and let $b = \mathcal{D}_\tau.\mathsf{query}(s, q)$. Then there exists a sequence π such that:

(1) π is a linear extension of (E, lo^C_E).

(2) $\mathsf{ops}(\pi) \cdot (q \Rightarrow b) \in \mathsf{Spec}$, which means that the operations in π, followed by the query, are admitted by the specification.

An execution $\tau \in [\![\mathcal{S}_{\mathcal{D}}]\!]$ is RA-linearizable with respect to Spec if all its configurations are RA-linearizable. The data type $\mathcal{D}_\tau$ is RA-linearizable with respect to Spec if all its executions are RA-linearizable.

4.2 Canonical States

Definition 4.5 considers concrete state of the active version at every replica, and requires the return value of any query operation on this state together with a linear extension of the update operations visible at the replica to satisfy the sequential specification. A natural way to establish this is to require that the concrete state of every version to be obtained by folding a linear extension of the update operations visible to the version. Then, we simply need to establish a simulation between the update operations in the implementation and the specification. However, since lo_E^C may not be a total order, it may have several linear extensions. We first show that folding every linear extension of (E, lo_E^C) reaches the same state.

Theorem 4.6 (One state per set of events). *Let $\mathcal{D}_\tau$ satisfy $\mathsf{Laws}(\mathcal{D}_\tau, \mathsf{Spec})$. Let C be a configuration and let $E \subseteq \mathcal{E}_C$ be causally closed. Then (E, lo_E^C) has at least one linear extension, and any two linear extensions π_1 and π_2 of (E, lo_E^C) produce the same state.*

$$\pi_1(\sigma_0) = \pi_2(\sigma_0).$$

The condition no-rc-chain ensures that lo_E^C is acyclic, so a linear extension exists. rc-non-comm and cond-comm ensure that swapping adjacent events which are unordered by lo_E^C does not change the final state. Since any two linear extensions can be obtained from one another by such swaps, they all produce the same state.

Definition 4.7 (Canonical state). Given a configuration C and a causally closed set $E \subseteq \mathcal{E}_C$, the *canonical state* of E, written $\mathsf{can}_C(E)$, is the state reached from σ_0 by applying the events of E in an order consistent with lo_E^C.

$$\mathsf{can}_C(E) = \pi(\sigma_0) \quad \text{for a linear extension } \pi \text{ of } (E, \mathsf{lo}_E^C).$$

For the empty event set, $\mathsf{can}_C(\emptyset) = \sigma_0$.

Definition 4.8 (Canonical correctness). A configuration $C = \langle S, H, P, \mathsf{vis}\rangle$ is *canonically correct* if every active version holds the canonical state of its event set.

$$S(v) = (s, E) \implies s = \mathsf{can}_C(E)$$

for every $v \in \mathsf{dom}(S)$.

Canonical correctness ensures the first clause of Definition 4.5. Together with Theorem 4.6, it also ensures Strong Eventual Consistency (SEC). If two replicas have the same event set at their head versions, then they will have the same linearization order, whose linear extension would lead to the same concrete state. We note that SEC is also an important correctness requirement for RDTs. In fact, we now show that our new definition of RA linearizability is critical in ensuring SEC as well, as the original definition of RA-linearizability in Soundarapandian et al. [2025] does not ensure SEC. Recall that the original definition requires a single linearization order over all update events of an execution, and then projects this order on the events visible to a version. In the example execution of Fig. 2, due to the absorber events R_p of A_p and R_q of A_q, the linearization order at version v_8 does not contain the rc relations $R_p \xrightarrow{\mathsf{rc}} A_q$ and $R_q \xrightarrow{\mathsf{rc}} A_p$. Now, if this same linearization order were to used for the version v_6, and then projected onto its events, the rc relation

$R_p \xrightarrow{\mathsf{rc}} A_q$ will also be removed at v_6. In this case, there are now 2 different linear extensions which fold to different concrete states: the sequence $[A_p, R_p, A_q]$ which folds to $\{(a, 2)\}$ and the sequence $[A_q, A_q, R_p]$ which folds to $\{\}$. Thus, the same event set can produce different states. Our definition rectifies this issue by allowing different linearization orders for the event sets at v_6 and v_8 (similarly at v_7 and v_8). Note that SEC alone does not guarantee that the replicas agree on a state that satisfies the sequential specification, which is taken care of by the second clause in Definition 4.5.

5 VERIFYING RA-LINEARIZABILITY

In this section, we describe a procedure to verify the newly proposed definition of RA-Linearizability for MRDTs. As mentioned in §4.2, we divide this in two steps: first we prove that every reachable configuration is canonically correct, which ensures that the state at every replica is the canonical state obtained by folding all the update events visible at the replica. In the second step, we will establish an equivalence between the update operations in the implementation and the specification.

The main difficulty in proving canonical correctness is in showing that a merge transition installs a canonical state at a replica. To show this, we first state five conditions on merge, called the *verification conditions VC1 - VC5*. Our overall proof strategy is to prove by induction on the length of an execution that every version in every reachable configuration contains the canonical state of its event set. Throughout, we assume that the laws in $\mathsf{Laws}(\mathcal{D}_\tau, \mathsf{Spec})$ from §4.1 hold.

Abbreviations. Throughout this section, C denotes a configuration and $\mathcal{E}_C$ its set of events, as defined in §4.1. Relations and predicates that depend on C are written with the corresponding subscripts and superscripts. We write

$$a \xrightarrow{\mathsf{vis}}_{\mathsf{NC}} b \quad \text{if and only if} \quad a \xrightarrow{\mathsf{vis}} b \wedge \neg(a \leftrightarrow b),$$

for the non-commuting visibility relation, and write $\xrightarrow{\mathsf{vis}}{}^{+}_{\mathsf{NC}}$ for its transitive closure. The *principal downset* of an event e is

$$\downarrow e = \{x \mid x = e \vee x \xrightarrow{\mathsf{vis}}{}^{+}_{\mathsf{NC}} e\}.$$

Thus, $\downarrow e$ contains e and every event that reaches e through a chain of non-commuting visibility edges. The events in the principal downset of e are guaranteed to be lo_E^C before e w.r.t. to any set E containing e.

An event e is *maximal* in E, written $\mathsf{Maximal}_E^C(e)$, if $e \in E$ and no event in E occurs after e in lo_E^C. A set E is *weakly closed* if for every event e in E, every visible predecessor that does not commute with e is also present in the set. Essentially, for every event $e \in E$, E must also contain its principal downset.

$$\mathsf{WeakClosed}_C(E) \iff \forall a, b.\ a \xrightarrow{\mathsf{vis}}_{\mathsf{NC}} b \wedge b \in E \Rightarrow a \in E.$$

Unlike causal closure, weak closure does not require commuting predecessors to be included. Such predecessors do not contribute to any edge in lo_E^C, so omitting them does not affect the ordering used to compute the canonical state.

We use two consequences of the assumed laws throughout the proof. First, NO-RC-CHAIN ensures that lo_E^C is acyclic. Hence every nonempty finite event set has a maximal event. Second, the laws ensure that all linear extensions of (E, lo_E^C) fold to the same state. We now state the verification conditions. Each one of them applies to canonical states of event sets that can arise during an execution. The predicate $\mathsf{Admissible}_C$ gathers the required assumptions for the VCs.

$$\mathsf{Admissible}_C(E_1, \ldots, E_n) \iff \bigwedge_{i=1}^{n} \big(E_i \subseteq \mathcal{E}_C \wedge \mathsf{WeakClosed}_C(E_i)\big).$$

VC1 (MERGE-COMM).

$$\forall l, a, b \in \Sigma. \quad \mathsf{merge}(l, a, b) = \mathsf{merge}(l, b, a).$$

VC1 states that swapping the two branch states does not change the result of a merge. It is the only verification condition that applies to arbitrary states. The remaining conditions apply to canonical states. For our running example, VC1 lets us choose which branch to call E_1 when applying VC3. In particular, the result of merge does not change irrespective of whether the replica p is merged into q (which is what happens in Fig. 2) or replica q is merged into p.

VC2 (MERGE-INIT).

$$\forall E \subseteq \mathcal{E}_C.\ \mathsf{Admissible}_C(E) \implies \mathsf{merge}(\sigma_0, \sigma_0, \mathsf{can}_C(E)) = \mathsf{can}_C(E).$$

VC2 handles the base case. When one branch has no events left, both that branch and the LCA have state σ_0. The merge must then return the state of the other branch unchanged.

VC3 (LOCAL-REDISTRIBUTE).

$$\begin{aligned}
&\forall E_1, E_2 \subseteq \mathcal{E}_C.\ \forall e \in E_1 \setminus E_2.\ \forall l, m, x, y \in \Sigma. \\
&\quad \mathsf{Admissible}_C(E_1, E_2) \wedge \mathsf{Maximal}^C_{E_1 \cup E_2}(e) \wedge l = \mathsf{can}_C(E_1 \cap E_2) \\
&\quad \wedge\ m = \mathsf{can}_C(\downarrow e \setminus \{e\}) \wedge\ x = \mathsf{can}_C(E_1 \setminus \{e\}) \wedge\ y = \mathsf{can}_C(E_2) \\
&\qquad \implies \mathsf{merge}\big(l,\ \mathsf{merge}(m, x, e(m)),\ y\big) = \mathsf{merge}\big(m,\ \mathsf{merge}(l, x, y),\ e(m)\big).
\end{aligned}$$

VC3 considers the case where none of the two branches being merged are in the initial state. In this case, following the bottom-up linearization strategy, we would like to first linearize or peel off a maximal event (say e) of the merged state. However, it is possible that no maximal event of the merged state would be a maximal event on either of the two branches being merged. Therefore, VC3 first performs a local re-distribution, by essentially changing the states being merged without changing the result of the merge to guarantee that e would be a maximal event on one of the two states.

VC3 is graphically illustrated in Fig. 4. To elaborate, let E_1 and E_2 be the event sets of the two branches being merged, and let e be a maximal event of the merged state, (i.e. $\mathsf{Maximal}^C_{E_1 \cup E_2}(e)$). Further, assume that $e \in E_1$ but $e \notin E_2$ (the case where e is in both E_1 and E_2 is handled by VC5). Now, the concrete state corresponding to the event set E_1 can itself be obtained by another merge: one branch would have every event of E_1 except e, while the other branch would have exactly the downset of e. This exactly corresponds to the merge on the LHS in the consequent of VC3.

In particular, l is the LCA state for the outer merge, y is the state corresponding to E_2, x is the state corresponding to $E_1 \setminus e$, and m is the state corresponding to the downset of e (excluding e itself). This is the merge that we want to linearize, and VC3 allows us to replace this with the merge on the RHS, where one of the branches being merged is exactly $e(m)$, which is guaranteed to have the event e as the maximal event. Note that this branch will contain the event e, while the other branch being merged contains the events $(E_1 \setminus \{e\}) \cup E_2$, thus leading to the same event set $E_1 \cup E_2$ for the merged state.

Essentially, VC3 has re-distributed the parameters of a nested merge so that the branch $e(m)$ which was appearing in an inner merge comes outside and becomes the parameter of the outer merge.

VC4 (CAUSAL-DELTA).

$$\begin{aligned}
&\forall E \subseteq \mathcal{E}_C.\ \forall e \in E.\ \forall A, B \in \Sigma. \\
&\quad \mathsf{Admissible}_C(E) \wedge \mathsf{Maximal}^C_E(e) \wedge A = \mathsf{can}_C(E \setminus \{e\}) \wedge B = \mathsf{can}_C(\downarrow e \setminus \{e\}) \\
&\qquad \implies \mathsf{merge}(B,\ A,\ e(B)) = e(A).
\end{aligned}$$

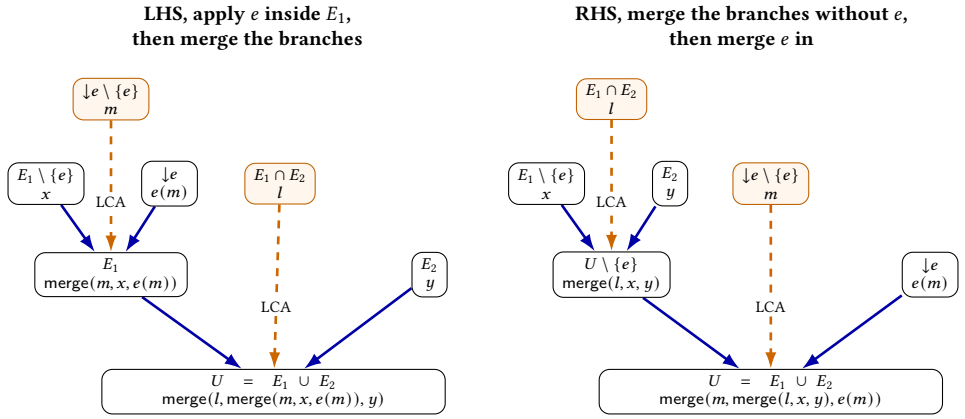


Fig. 4. The two sides of VC3, where the maximal event e lies in E_1 only. Each box carries an event set above its canonical state, and the arrows into a box are the three arguments of one merge, the dashed one being the LCA. VC3 says the two boxes at the bottom are equal.

VC4 takes over from VC3 and does the actual bottom-up linearization by peeling off the maximal event of the merged state. In particular, it's pre-conditions require that the two branches being merged are exactly those from the RHS of the merge in VC3: e is the maximal event of the merge result, E is the entire set of events in the merge result, the first branch (state A) corresponds to $E \setminus \{e\}$ and the second branch (state B) is simply the downset of e. Notice that the RHS of VC4 replaces the merge with a simple application of e, thus linearing e at the end.

VC5 (REDISTRIBUTE).

$$\begin{aligned}
&\forall E_1, E_2 \subseteq \mathcal{E}_C.\ \forall e \in E_1 \cap E_2.\ \forall m, x_0, x_1, x_2 \in \Sigma.\\
&\quad \mathsf{Admissible}_C(E_1, E_2) \wedge \mathsf{Maximal}^C_{E_1 \cup E_2}(e) \wedge m = \mathsf{can}_C(\downarrow e \setminus \{e\})\\
&\quad \wedge\ x_0 = \mathsf{can}_C((E_1 \cap E_2) \setminus \{e\}) \wedge x_1 = \mathsf{can}_C(E_1 \setminus \{e\}) \wedge x_2 = \mathsf{can}_C(E_2 \setminus \{e\})\\
&\qquad \Longrightarrow\ \mathsf{merge}\big(\mathsf{merge}(m, x_0, e(m)),\ \mathsf{merge}(m, x_1, e(m)),\ \mathsf{merge}(m, x_2, e(m))\big)\\
&\qquad\quad = \mathsf{merge}\big(m,\ \mathsf{merge}(x_0, x_1, x_2),\ e(m)\big).
\end{aligned}$$

VC5 is the counterpart of VC3 in the case when the maximal event in the final merged state belongs to both branches and their LCA. The pre-conditions in VC5 ensure this, and in the post-condition, the merge on the LHS is that actual merge, whose branches are re-written as inner merges which separate out the maximal event e and its downset. The RHS then performs the re-distribution by moving e along with its downset as a branch of the outer merge.

End to End example: To illustrate the conditions, we will use the execution in Fig. 2, in particular the merge which leads to the version v_8. The two versions being merged, v_6 and v_7, have event sets

$$E_1 = \{A_p, R_p, A_q\}, \qquad E_2 = \{A_q, R_q, A_p\}, \qquad E_1 \cap E_2 = \{A_p, A_q\}.$$

Let $U = E_1 \cup E_2$. Note that all of these event sets are weakly closed. The merging states are

$$\mathsf{can}_C(E_1 \cap E_2) = \{(a,1),(a,2)\}, \qquad \mathsf{can}_C(E_1) = \{(a,2)\}, \qquad \mathsf{can}_C(E_2) = \{(a,1)\}.$$

We therefore need to show that $\mathsf{merge}\big(\{(a,1),(a,2)\}, \{(a,2)\}, \{(a,1)\}\big) = \mathsf{can}_C(U) = \{\}$.

Within U, the only edges in lo_U^C are $A_p \xrightarrow{\mathsf{vis}} R_p$ and $A_q \xrightarrow{\mathsf{vis}} R_q$, since each add_a has an absorber in U. We will compute the canonical state by peeling off maximal events in the order R_p, A_p, R_q, and A_q. Each verification condition justifies one of the steps in this reduction.

First, we apply VC3. Note that both R_p and R_q are maximal in U, but neither of the two are maximal in v_6 or v_7. We choose $e = R_p$, which belongs to E_1 only. Thus, all the pre-conditions of VC3 are met.

The four states in VC3 (i.e. l, x, y, m) as shown in Fig. 4 now become the following:

$$m = \mathsf{can}_C(\downarrow R_p \setminus \{R_p\}) = \{(a,1)\}, \qquad x = \mathsf{can}_C(E_1 \setminus \{R_p\}) = \{(a,1),(a,2)\},$$
$$l = \mathsf{can}_C(E_1 \cap E_2) = \{(a,1),(a,2)\}, \qquad y = \mathsf{can}_C(E_2) = \{(a,1)\}.$$

The state m is the state observed by R_p, it contains the element added by A_p, and applying R_p gives $R_p(m) = \{\}$. The state x is the first branch with R_p removed, while y is the second branch. The actual merge performed is the left-hand side of VC3, as shown below, where the LHS is the merge leading to the version v_8, and the RHS is the LHS of VC3:

$$\mathsf{merge}(\mathsf{can}_C(E_1 \cap E_2), \mathsf{can}_C(E_1), \mathsf{can}_C(E_2)) = \mathsf{merge}(l, \mathsf{merge}(m, x, R_p(m)), y).$$

l and y are defined to be ($\mathsf{can}_C(E_1 \cap E_2)$ and $\mathsf{can}_C(E_2)$), while $\mathsf{can}_C(E_1)$ is rewritten as a merge between the events of E_1 minus R_p (corresponding to the state x) and the downset of R_p. VC3 allows us to rewrite the RHS above as:

$$\mathsf{merge}\big(m, \mathsf{merge}(l, x, y), R_p(m)\big).$$

Next, we apply VC4 which replaces the above merge with the application of R_p to the first branch state. That is:

$$R_p(\mathsf{merge}(l, x, y))$$

Having peeled off R_p, the remaining event set for $\mathsf{merge}(l, x, y)$ is $U \setminus \{R_p\} = \{A_p, A_q, R_q\}$. The two branch event sets corresponding to the state x and y are

$$F_1 = E_1 \setminus \{R_p\} = \{A_p, A_q\}, \qquad F_2 = E_2 = \{A_q, R_q, A_p\}.$$

We recompute the ordering for their union. With R_p removed, A_p no longer has an absorber in the event set, so the edge $R_q \xrightarrow{\mathsf{rc}} A_p$ remains. Thus, A_p is maximal and belongs to both F_1 and F_2. Hence, VC5 applies to this case.

Taking $e = A_p$, the four states (m, x_0, x_1, x_2) used by VC5 are given below:

$$m = \mathsf{can}_C(\emptyset) = \sigma_0, \qquad x_0 = \mathsf{can}_C((F_1 \cap F_2) \setminus \{A_p\}) = \{(a,2)\},$$
$$x_1 = \mathsf{can}_C(F_1 \setminus \{A_p\}) = \{(a,2)\}, \qquad x_2 = \mathsf{can}_C(F_2 \setminus \{A_p\}) = \{\}.$$

The event A_p observed the empty state, so $A_p(m) = \{(a,1)\}$. The states x_0, x_1, and x_2 are the ancestor and branch states after removing A_p. $\mathsf{merge}(l, x, y)$ can now be rewritten as the LHS of VC5:

$$\mathsf{merge}(l, x, y) = \mathsf{merge}\big(\mathsf{merge}(m, x_0, e(m)),\ \mathsf{merge}(m, x_1, e(m)),\ \mathsf{merge}(m, x_2, e(m))\big)$$

On the LHS of VC5, the effect of $e = A_p$ is included in each of the three states before the outer merge. On the right-hand side, the states are first merged without A_p, and its effect is then included through an outer merge, allowing us to rewrite the RHS above as:

$$\mathsf{merge}\big(m, \mathsf{merge}(x_0, x_1, x_2), A_p(m)\big).$$

By VC4, we can now replace the above merge with the application of Ap to the inner merge:

$$A_p(\mathsf{merge}(x_0, x_1, x_2))$$

Thus, the peeling off of A_p is complete. The two remaining events corresponding to $\mathsf{merge}(x_0, x_1, x_2)$ are A_q and R_q. The event R_q is maximal and belongs to only one branch, so VC1 allows us to exchange the branches and apply VC3. VC4 then completes the step, giving

$$R_q(\{(a, 2)\}) = \{\}.$$

Only A_q remains. Once the other branch and the LCA have reached σ_0, VC2 gives

$$\mathsf{merge}(\sigma_0, \sigma_0, \mathsf{can}_C(\{A_q\})) = \mathsf{can}_C(\{A_q\}) = A_q(\sigma_0) = \{(a, 2)\}.$$

Together, peeling off all the four event applications gives us

$$\mathsf{can}_C(U) = R_p\Big(A_p\big(R_q\big(A_q(\sigma_0)\big)\big)\Big) = \{\}.$$

We call VC1- VC5 the *canonical join laws*. The verification conditions are about individual merges, whereas the theorem below establishes canonical correctness for every version in every reachable configuration.

Theorem 5.1. *If an implementation satisfies the canonical join laws stated above, then every reachable configuration is canonically correct.*

We prove the theorem by induction on the length of the execution, maintaining canonical correctness for every version. The initial configuration C_0 is canonically correct because its only version holds σ_0, the canonical state of the empty event set. For Query, Fork and Apply steps, the proof is straightforward. For Merge step, the canonical join laws guarantee the following join lemma which guarantees that merge of canonical states results in a new canonical state.

Lemma 5.2 (Join). *Let $\mathcal{D}_\tau$ satisfy the canonical join laws, and let C be a configuration. Then, for all supported and weakly closed $E_1, E_2 \subseteq \mathcal{E}_C$,*

$$\mathsf{can}_C(E_1 \cup E_2) = \mathsf{merge}\big(\mathsf{can}_C(E_1 \cap E_2),\ \mathsf{can}_C(E_1),\ \mathsf{can}_C(E_2)\big).$$

6 CORRECTNESS AGAINST A SEQUENTIAL SPECIFICATION

Theorem 5.1, together with Theorem 4.6, establishes SEC through canonical correctness. However, canonical correctness alone does not guarantee RA-linearizability. Since the canonical state $\mathsf{can}_C(E)$ is computed using the implementation's own update operation, an incorrect update operation can cause all replicas to converge to the same incorrect state. Canonical correctness does not establish whether the resulting behaviour agrees with the sequential specification. We illustrate this with an implementation that converges but violates its sequential specification.

A convergent implementation that is not a set. Consider the OR-set of Fig. 1, modified so that removal operations leave the state unchanged instead of removing the element.

$$\mathsf{do}(\sigma, _, _, \mathsf{rem}_a) = \sigma.$$

All other parts of the implementation remain unchanged. Since removals are identity operations and additions of distinct entries commute, every pair of operations commutes. So, rc is empty, and the first three conditions of $\mathsf{Laws}(\mathcal{D}_\tau, \mathsf{Spec})$ hold. The merge is unchanged, so VC1 through VC5 continue to hold as well. Thus, every version holds the canonical state of its event set, and replicas that have observed the same events have the same state.

But the implementation violates its sequential specification. Suppose a replica executes add_a, followed by rem_a, and then reads the set. Since removal has no effect, the resulting state is $\{(a, 1)\}$,

and rd_a returns true. In the natural specification, the same sequence results in $\{\}$, so the read returns false. It also violates SPEC-NON-COMM in §4.1, because add_a and rem_a commute in the incorrect implementation but not in Spec.

The following theorem establishes RA-linearizability by combining canonical correctness with sequential conformance for the linear extensions of each version's event set.

THEOREM 6.1. *Let $\mathcal{D}_\tau$ satisfy $\mathsf{Laws}(\mathcal{D}_\tau, \mathsf{Spec})$. Consider a reachable configuration $C = \langle S, H, P, \mathsf{vis}\rangle$, and suppose every version of C holds the canonical state of its event set, and for every version $v \in \mathsf{dom}(S)$ with $S(v) = (s, E)$, every linear extension π of (E, lo_E^C), and every query $q \in Q_\tau$,*

$$\mathsf{ops}(\pi) \cdot \big(q \Rightarrow \mathcal{D}_\tau.\mathsf{query}(\pi(\sigma_0), q)\big) \in \mathsf{Spec}.$$

Then C is RA-linearizable with respect to Spec*, in the sense of Definition 4.5.*

To prove the theorem, consider an active version v with $S(v) = (s, E)$. By canonical correctness, there exists a linear extension π of (E, lo_E^C) such that

$$\pi(\sigma_0) = \mathsf{can}_C(E) = s.$$

This establishes the first clause of Definition 4.5. By the premise of the theorem applied to π and q, the same sequence, followed by any query $q \in Q_\tau$, produces behaviour permitted by Spec. This establishes the second clause, and hence C is RA-linearizable with respect to Spec.

Proving sequential conformance. We prove sequential conformance by induction on the length of an event sequence. Let $\mathsf{Spec} = \langle L, \Phi, \phi_0, \rightarrow\rangle$ be the specification, as defined in Definition 4.4. We use an *abstraction function* $\alpha : \Sigma \rightarrow \Phi$ to relate implementation states to specification states. The function must satisfy the following conditions:

(i) $\alpha(\sigma_0) = \phi_0$.

(ii) For every state $\sigma \in \Sigma$, timestamp t, replica r, and update $o \in O_\tau$, $\alpha(\sigma) \xrightarrow{o} \alpha(\mathsf{do}(\sigma, t, r, o))$.

(iii) For every state $\sigma \in \Sigma$ and every query $q \in Q_\tau$, and value $b \in Val_\tau$, if $\alpha(\sigma) \xrightarrow{q \Rightarrow b} \alpha(\sigma)$, then $\mathcal{D}_\tau.\mathsf{query}(\sigma, q) = b$.

Condition (i) makes α hold at the initial states, and condition (ii) makes each update preserve it. Hence, by induction on the length of an event sequence π,

$$\alpha(\pi(\sigma_0)) = \mathsf{ops}(\pi)(\phi_0).$$

This means that applying the abstraction function to the implementation state gives the same specification state as executing the corresponding operations directly in the specification. Condition (iii) ensures that queries return the same results in the implementation and the specification. Together, the three conditions establish the sequential conformance required by Theorem 6.1.

For the OR-set of Fig. 1, consider the abstraction function

$$\alpha(\sigma) = \{a \mid \exists k.\ (a, k) \in \sigma\}.$$

It removes the timestamps from the implementation state and retains only the elements in the set. Since both systems start with the empty set, condition (i) holds.

To verify condition (ii), consider the two update operations. An add_a operation inserts (a, t) into the implementation state, so a is included in the abstract state. An rem_a operation removes all entries associated with a, so a is not present in the state. These updates match the specification transitions

$$\phi \xrightarrow{\mathsf{add}_a} \phi \cup \{a\} \qquad \text{and} \qquad \phi \xrightarrow{\mathsf{rem}_a} \phi \setminus \{a\},$$

as defined in §2.2. Thus, condition (ii) holds for both operations.

For condition (iii), the query rd_a returns $\exists k.\ (a,k) \in \sigma$. By the definition of α, this is equivalent to checking whether $a \in \alpha(\sigma)$. The implementation therefore returns the same Boolean result as the specification transition $\phi \xrightarrow{\mathsf{rd}_a \Rightarrow b} \phi$ at $\phi = \alpha(\sigma)$. Hence, all three conditions hold for the OR-set implementation.

The modified implementation, in which removals have no effect, does not satisfy condition (ii). Consider the sequence $[\mathsf{add}_a, \mathsf{rem}_a]$. The specification reaches $\{\}$, but the implementation retains the state $\{(a,1)\}$. Consequently, the implementation returns true for rd_a, whereas the specification returns false.

Moreover, this failure cannot be avoided by choosing a different abstraction function. Let $\sigma = \{(a,1)\}$ be the implementation state reached by executing add_a from σ_0. Condition (i) fixes $\alpha(\sigma_0) = \{\}$, and condition (ii), applied to add_a, requires $\alpha(\sigma) = \{a\}$. Now apply rem_a to σ. The implementation state remains unchanged, but the specification moves from $\{a\}$ to $\{\}$. Condition (ii) would therefore also require $\alpha(\sigma) = \{\}$. This contradicts $\alpha(\sigma) = \{a\}$. Thus, no abstraction function can satisfy all three conditions for the modified implementation.

7 ISSUANCE GUARDS AND EXECUTION-CERTIFIED MRDTS

Theorem 6.1 requires the specification to admit every linear extension of an event set at every version. This holds for the OR-set, where every operation is defined at every state, but not for data types such as lists. For example, in a list data type such as the Replicated Growable Array (RGA) of Roh et al. [2011], an insertion may add an element after an anchor that has not been added yet, or a deletion may remove an element that is not present. Such sequences violate the specification, even though they cannot arise in a valid execution. Events are generated at replica head versions, so their arguments are determined by the state at those versions. The Apply rule of §3 does not impose this restriction, it constrains only the timestamp and can therefore produce configurations containing events that no replica could have generated. We capture the conditions for generating events with an issuance guard.

Definition 7.1 (Issuance guard and certified executions). An *issuance guard* is a predicate $G(e,s)$ stating whether event e may be generated at state s. A configuration C is *issuance-certified* if, for every $e \in \mathcal{E}_C$, there is a linear extension π_e of $(E_e, \mathsf{lo}^C_{E_e})$, where $E_e = \{e' \in \mathcal{E}_C \mid e' \xrightarrow{\mathsf{vis}} e\}$, such that $G\big(e, \pi_e(\sigma_0)\big)$. An execution is *$G$-certified* if it starts from the initial configuration and every configuration it reaches is issuance-certified.

The guard is checked when an event is generated, not again when a linearization is folded, since folding passes through states that the issuing replica never observed. We can therefore weaken the premise of Theorem 6.1 in two ways. It only needs to hold for versions of certified configurations, and it only needs to hold for one linear extension of each event set, rather than all of them.

Definition 7.2 (Certified history adequacy). Fix an issuance guard G. The implementation and Spec satisfy *certified history adequacy* if, for every G-certified configuration C, every version with state and event set (s,E), and every query $q \in Q_\tau$, there is a linear extension π of (E, lo^C_E) such that

$$\mathsf{ops}(\pi) \cdot \big(q \Rightarrow \mathcal{D}_\tau.\mathsf{query}(\pi(\sigma_0), q)\big) \in \mathsf{Spec}.$$

Theorem 7.3. *Suppose $\mathcal{D}_\tau$ satisfies $\mathsf{Laws}(\mathcal{D}_\tau, \mathsf{Spec})$, the canonical join laws of §5, and certified history adequacy for an issuance guard G. Then every configuration of every G-certified execution is RA-linearizable with respect to* Spec.

Example 7.4 (RGA). An addAfter operation takes the identity of the anchor after which the new element is inserted. Its own fresh timestamp becomes the identity of the inserted element. A remove operation takes the identity of the element to be removed.

Let the sequential state be a pair (A, T), where A is the list of identities that have been added and T is the set of removed identities. The initial state is $([0], \varnothing)$, where 0 is a reserved root. We write $\mathrm{after}(A, \ell, t)$ for the list obtained by inserting t immediately after ℓ in A. The specification is

$$(A,T) \xrightarrow{\mathsf{addAfter}(t,\ell)} (\mathrm{after}(A,\ell,t),\ T) \quad [\, t \neq 0 \wedge t \notin A \wedge \ell \in A \,]$$
$$(A,T) \xrightarrow{\mathsf{remove}(x)} (A,\ T \cup \{x\}) \quad [\, x \in A \,]$$
$$(A,T) \xrightarrow{\mathsf{rd} \Rightarrow L} (A,T) \quad [\, L = A \setminus (T \cup \{0\}) \,]$$

Here $A \setminus S$ removes the identities of S from the list A while retaining the order of the elements. The issuance guard requires an inserted identity to be fresh and nonzero, an insertion anchor to be present at the issuing replica, and a deleted identity to have been added already.

We now show that these conditions ensure certified history adequacy. Consider a version with event set E, and let π be any linear extension of (E, lo_E^C). Suppose an event e adds an identity t after an anchor ℓ. Since ℓ was present when e was created, an earlier event a must have added it. The event a is visible to e, and causal closure ensures that $a \in E$. Moreover, inserting an element after its anchor does not commute with the operation that added that anchor. By the first clause of Definition 4.1, we therefore have $a\ \mathsf{lo}_E^C\ e$. Hence, a precedes e in π, and the anchor exists when the insertion is applied. The same reasoning applies to deletion. The event that added the deleted element precedes the deletion in π. Finally, the issuance guard ensures that inserted identities are fresh and nonzero. Thus, every operation in π satisfies its preconditions, and certified history adequacy follows.

8 IMPLEMENTATION AND CASE STUDIES

Mechanization. We have implemented the framework and its metatheory in Lean 4 [de Moura and Ullrich 2021]. The development defines the replicated store, event orders, sequential history languages, and RA-linearizability. It checks canonical-state determinacy, the derivation of the join lemma from the five VCs, execution adequacy, and the separate sequential bridge to RA-linearizability, including recursively computed merge bases. These proofs formalize the sufficient-condition argument of §5–§7, with the representation and replay hypotheses described below. The history witness satisfies the required event order and specification visibility and explains the implementation's query result.

The selected generic theorem dependencies comprise 4,020 lines of Lean theorem statements and proofs, with a further 1,688 lines proving the shared automation templates sound. These counts exclude blank lines, comments, datatype-specific proofs, and tests; shared declarations are counted once. We use Lean 4.28.0 and Mathlib. The audited theorem endpoints depend only on Lean's standard logical axioms. These results are proved with concrete equality of implementation states. Representation invariants restrict the states to which a law applies; the relation to the sequential specification is used separately in the sequential bridge. The proof interfaces make representation uniqueness, replay supply, and dependency closure explicit. These are premises of the sufficient-condition theorem, discharged by each instantiation, rather than consequences of the five merge equations alone. The artifact records the correspondence between the paper's statements and the Lean declarations.

Verified data types. Table 1 reports 26 configurations with checked RA-linearizability results. We display them in 24 rows, grouping the three representations of a grow-only set. Rows distinguish implementations and query interfaces, not distinct abstract data types: for example, the rich-text core and rendered query share merge proofs. We verify explicitly defined models and adaptations

Table 1. Verified configurations and current Lean source footprint. MRDT and Spec count definition dependencies; VCs and Bridge count proof-support dependencies, including theorem statements, helper definitions, and certificate wiring. Shared framework libraries are excluded.

Configuration	MRDT	Spec	VCs	Bridge	I	E	rc ≠ ∅
Grow-only set [Shapiro et al. 2011a]	40	15	70	166	–	–	–
Counter [Soundarapandian et al. 2025]	18	5	58	97	–	–	–
Grow-only map [Shapiro et al. 2011a]	14	5	55	100	–	–	–
LWW register [Shapiro et al. 2011a]	30	7	65	82	–	Y	–
RGA [Roh et al. 2011]	54	19	55	910	Y	Y	–
OR-set [Soundarapandian et al. 2025]	26	20	56	256	–	–	Y
Efficient OR-set [Soundarapandian et al. 2025]	27	18	72	1,092	–	–	Y
Multivalue register [Shapiro et al. 2011a]	31	11	71	918	Y	Y	–
Embedded RGA [Roh et al. 2011]	57	42	129	2,422	Y	Y	–
Sided RGA [Roh et al. 2011]	66	48	137	3,102	Y	Y	–
Peritext (boundaries) [Litt et al. 2022]	69	43	130	2,400	Y	Y	–
Sided Peritext (core) [Litt et al. 2022]	117	55	134	2,870	Y	Y	–
Sided Peritext (rendered) [Litt et al. 2022]	206	60	146	2,957	Y	Y	–
Anchored queue [Roh et al. 2011; Soundarapandian et al. 2022]	71	26	135	2,606	Y	Y	–
FugueMax [Weidner and Kleppmann 2025]	162	38	386	6,738	Y	Y	–
Bounded counter [Balegas et al. 2015]	51	35	133	501	Y	Y	–
Enable-wins flag [Soundarapandian et al. 2025]	35	11	532	582	–	–	Y
Disable-wins flag [Soundarapandian et al. 2025]	35	11	532	582	–	–	Y
Tree move (replay) [Kleppmann et al. 2022]	56	15	65	94	Y	Y	–
AegisSheet (intent model) [Pfeil et al. 2026]	336	140	53	2,173	Y	Y	–
Notion core [Nehmzow et al. 2026]	316	112	153	15,288	Y	Y	–
Remove-wins set [Soundarapandian et al. 2025]	28	18	434	168	Y	–	Y
Set-wins multimap [Soundarapandian et al. 2025]	25	12	19	136	–	–	Y
Optional register [Soundarapandian et al. 2025]	61	9	717	880	Y	Y	Y

I: a nontrivial datatype-specific issuance guard. E: an execution-scoped history bridge, rather than conformance of every implementation fold. These execution facts are proved from reachability, not supplied as unproved invariants. The final column concerns the application-operation policy; an empty policy does not remove origin, visibility, or invariant-scoped ordering constraints. All rows have ordinary and recursive-base results.

Counts exclude blank lines, comments, tests, and shared framework code. Each row assigns source lines once, in column order; Bridge includes the remaining execution and issuance proof support. Dependencies reused across rows are counted in each row, so column sums are not project totals. The artifact supplies the source roots, hashes, and counting script.

of the cited algorithms. For AegisSheet, the model captures the published spreadsheet intent semantics; it is not a correctness claim for the associated Scala implementation.

Both OR-set implementations use the same ordinary-set sequential specification. The bounded counter supports transferring rights between replicas as well as increment and decrement. For identified sequence operations, the sequential specifications retain the identities needed to interpret insertion anchors and deletion targets. Additional local refinement theorems connect positional client edits to these identified operations. Neem's set-wins map retains multiple values at a key; we supply a set-valued lookup and verify an accumulating multimap specification. The remove-wins set reserves timestamp zero for its initial removal tags, and the optional register preserves Neem's source-state refinement; both use a positive-timestamp issuance guard.

Proof effort and automation. The two proof columns distinguish establishing the five merge VCs from establishing the sequential bridge. We provide reusable VC templates for commuting updates, policy-directed updates, and certified record representations. Lean's simplification and proof-search tactics discharge the resulting finite obligations using registered implementation laws. The template soundness proofs derive the original five VCs, so the automation does not enlarge the trusted base. Datatype-specific representation and issuance arguments, and the sequential bridge, remain part of the proof effort.

Notion core. The Notion case study starts from the publicly available JavaScript demo accompanying Notion's account of collaborative editing [Nehmzow et al. 2026; Notion 2026]. We used the demo as an executable oracle: we issued editing requests, exchanged the generated transactions, and inspected the resulting rendered documents. We pinned the source files and preserved the request traces so that the observations can be reproduced. Guided by the source and these observations, we formalized a typed model of block creation, insertion, deletion, and splitting operations.

The MRDT stores immutable transactions and merges them by set union. Its query reconstructs and renders a document using the transactions' identifiers and structural dependencies. The five merge VCs therefore reuse the finite-set development, but convergence of this carrier does not establish correctness of the rendered document. The substantial part of the proof relates text instances and slice trees to an independent sequential specification: an ordered list of blocks containing retained character and boundary identities. This specification contains neither a transaction log nor the implementation's replay algorithm or slice tree.

We prove RA-linearizability for the typed core under an explicit client protocol with fresh session identities, initialization, and locally checked issuance. The proof derives allocation freshness, the presence of referenced origins, and the partition invariants needed to interpret splits. For every stored version, one history supplies the required order, specification visibility, and the same query result as the implementation. Local refinement theorems also show that prepared insertions, deletions, and splits have the requested positional effect. The result covers ordinary and recursively computed Sal merge bases. The model is source-audited and tested against the pinned JavaScript; there is no extraction theorem or claim about Notion's production implementation.

Bugs in the public demo. The oracle experiments also found three failures of ordinary editing behaviour in the broader demo. After concurrent splits and a block-merge request, bolding the two displayed characters in one block can bold neither of them. In a second trace, bolding a block also bolds an unselected character in another block. A third trace reaches a block containing `aX`; splitting it at position zero produces `X` and `a`, rather than an empty block followed by `aX`. All three traces use requests generated by the demo's own client. They expose a mismatch between displayed order and the underlying text or subtree intervals. The artifact contains executable regressions and Lean counterexamples for the relevant sequential contracts. Formatting and the editor's block-merge operation are outside the verified core; block merging here is an editing request, distinct from merging two replicated versions.

9 RELATED WORK

10 CONCLUSION

REFERENCES

Valter Balegas, Diogo Serra, Sérgio Duarte, Carla Ferreira, Marc Shapiro, Rodrigo Rodrigues, and Nuno Preguiça. 2015. Extending Eventually Consistent Cloud Databases for Enforcing Numeric Invariants. In *2015 IEEE 34th Symposium on Reliable Distributed Systems*. IEEE, 31–36. https://doi.org/10.1109/SRDS.2015.32

Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction—CADE 28*. Springer, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37

Git. 2021. Git: A distributed version control system. https://git-scm.com/

Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Trans. Program. Lang. Syst.* 12, 3 (1990), 463–492. https://doi.org/10.1145/78969.78972

Irmin. 2021. Irmin: A distributed database built on the principles of Git. https://irmin.org/

Martin Kleppmann, Dominic P. Mulligan, Victor B. F. Gomes, and Alastair R. Beresford. 2022. A Highly-Available Move Operation for Replicated Trees. *IEEE Transactions on Parallel and Distributed Systems* 33, 7 (July 2022), 1711–1724. https://doi.org/10.1109/TPDS.2021.3118603

Leslie Lamport. 1978. Time, Clocks, and the Ordering of Events in a Distributed System. *Commun. ACM* 21, 7 (1978), 558–565. https://doi.org/10.1145/359545.359563

Geoffrey Litt, Sarah Lim, Martin Kleppmann, and Peter van Hardenberg. 2022. Peritext: A CRDT for Collaborative Rich Text Editing. *Proceedings of the ACM on Human-Computer Interaction* 6, CSCW2, Article 531 (Nov. 2022), 36 pages. https://doi.org/10.1145/3555644

Angelique Nehmzow, Emma Guo, Fabricio Pontes Harsich, and Stephan Boyer. 2026. How Notion handles concurrent editing with CRDTs. Notion engineering blog. https://www.notion.com/blog/how-notion-handles-concurrent-editing-with-crdts Published 18 September 2026; accessed 14 October 2026.

Notion. 2026. Notion CRDT interactive demo. https://www.notion.com/front-static/crdt-demo/index.html?embed=1 Public educational demo; seven module URLs and SHA-256 hashes pinned in the Sal artifact's sources.json, observed 14 October 2026.

Florian Pfeil, David Scandurra, and Julian Haas. 2026. AegisSheet: A Compositional CRDT for Collaborative Spreadsheets. In *Proceedings of the 13th International Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC '26)*. Association for Computing Machinery, 40–46. https://doi.org/10.1145/3806077.3806695

Hyun-Gul Roh, Myeongjae Jeon, Jin-Soo Kim, and Joonwon Lee. 2011. Replicated Abstract Data Types: Building Blocks for Collaborative Applications. *J. Parallel Distributed Comput.* 71, 3 (2011), 354–368. https://doi.org/10.1016/j.jpdc.2010.12.006

Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011a. *A comprehensive study of Convergent and Commutative Replicated Data Types*. Technical Report RR-7506. Inria – Centre Paris-Rocquencourt; INRIA. 50 pages. https://hal.inria.fr/inria-00555588

Marc Shapiro, Nuno M. Preguiça, Carlos Baquero, and Marek Zawirski. 2011b. Conflict-Free Replicated Data Types. In *Stabilization, Safety, and Security of Distributed Systems - 13th International Symposium, SSS 2011, Grenoble, France, October 10-12, 2011. Proceedings (Lecture Notes in Computer Science, Vol. 6976)*. Springer, 386–400. https://doi.org/10.1007/978-3-642-24550-3_29

Vimala Soundarapandian, Adharsh Kamath, Kartik Nagar, and KC Sivaramakrishnan. 2022. Certified Mergeable Replicated Data Types. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '22)*. Association for Computing Machinery. https://doi.org/10.1145/3519939.3523735

Vimala Soundarapandian, Kartik Nagar, Aseem Rastogi, and KC Sivaramakrishnan. 2025. Automatically Verifying Replication-Aware Linearizability. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 111 (April 2025), 27 pages. https://doi.org/10.1145/3720452

Chao Wang, Constantin Enea, Suha Orhun Mutluergil, and Gustavo Petri. 2019. Replication-aware linearizability. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019, Phoenix, AZ, USA, June 22-26, 2019*. ACM, 980–993. https://doi.org/10.1145/3314221.3314617

Matthew Weidner and Martin Kleppmann. 2025. The Art of the Fugue: Minimizing Interleaving in Collaborative Text Editing. *IEEE Transactions on Parallel and Distributed Systems* 36, 11 (Nov. 2025), 2425–2437. https://doi.org/10.1109/TPDS.2025.3611880